Tutorial for geogxboost Python library

last update 21/03/2025

This tutorial provides a practical guide to applying the geoxgboost library (Grekousis, 2025) in geographical analysis workflows. You can use it with any Python IDE, and it was created using PyCharm 2023.3.3 (Community Edition).

Useful links

For hands-on experience, demo data and a tutorial are available on GitHub: https://github.com/geogreko/DemoGXGBoost

Comprehensive documentation, including function details, parameter descriptions, and usage examples for the geoxgboost package, can be found on Read the Docs: https://geoxgboost.readthedocs.io/en/latest/

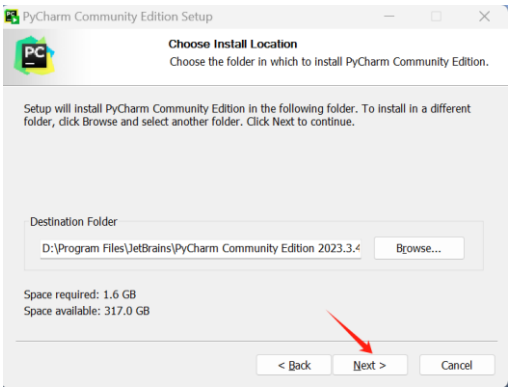
The theoretical foundations of the Geographical-XGBoost algorithm are presented in the following publication: https://doi.org/10.1007/s10109-025-00465-4

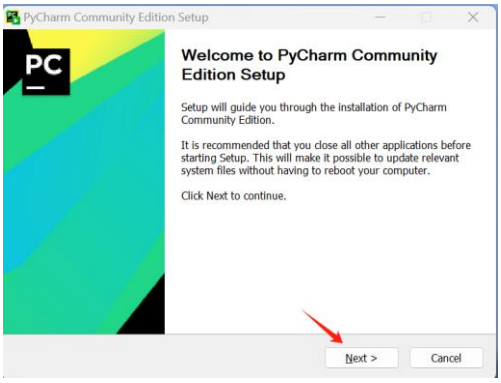
PyCharm quick guide installation

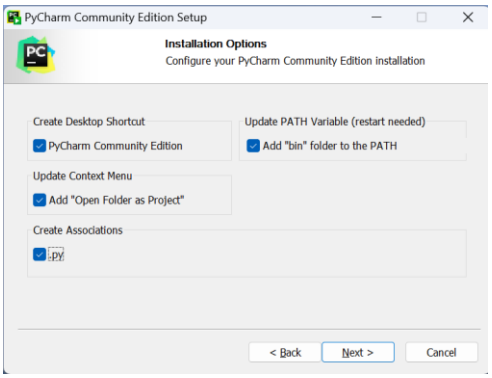
To download and install PyCharm go to:

https://www.jetbrains.com/pycharm/download/?section=windows and select Community Edition.

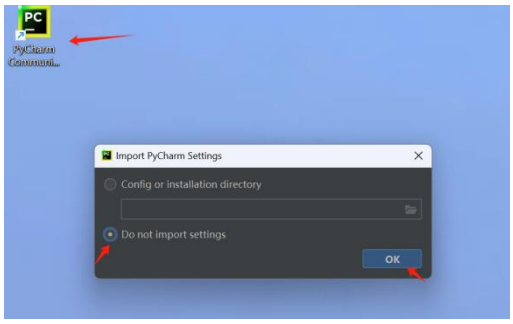
Install PyCharm > Next > Next

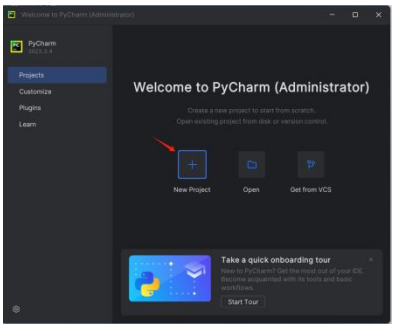






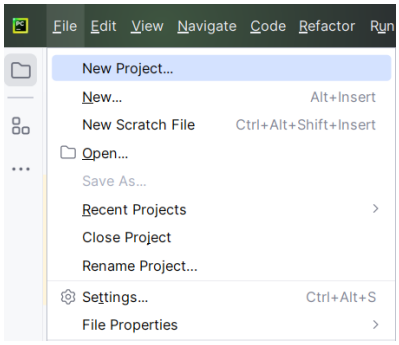
Finsh (reboot later) > Open PyCharm for the first time > Click New Project



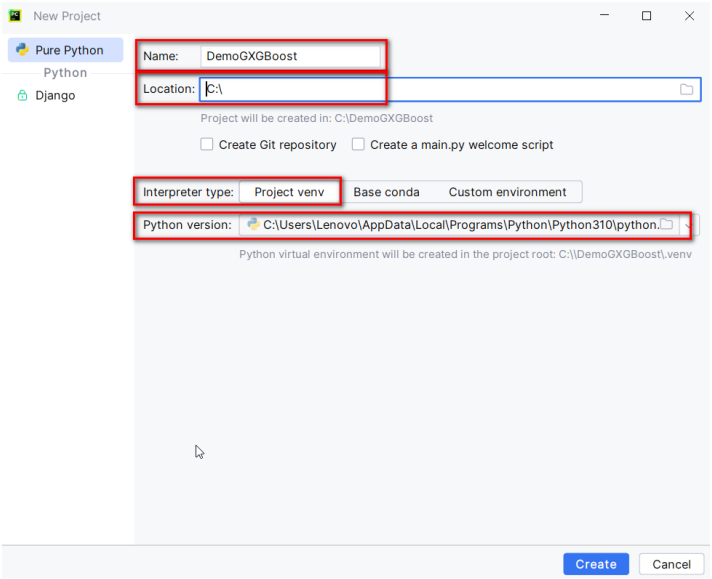


Create project

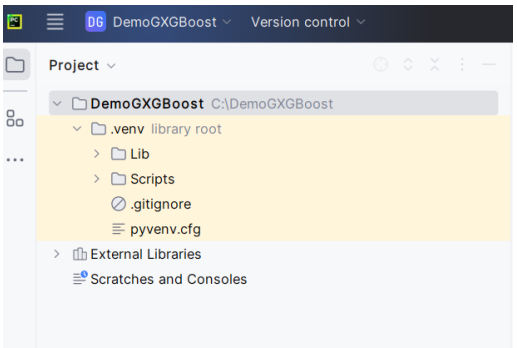
Alternatively, you and create a new project through the main tab (in case you have already installed PyCharm



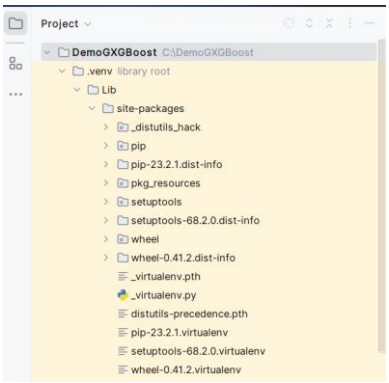
- File->New Project
- Set the name of the project and the location as in the right image
- Keep the **Interpreter type** to **Project venv**
- The system will select the Python version that has already been installed. If there is no Python installation, PyCharm will automatically install the latest version
- Click **Create**



The new project and virtual environment are created.

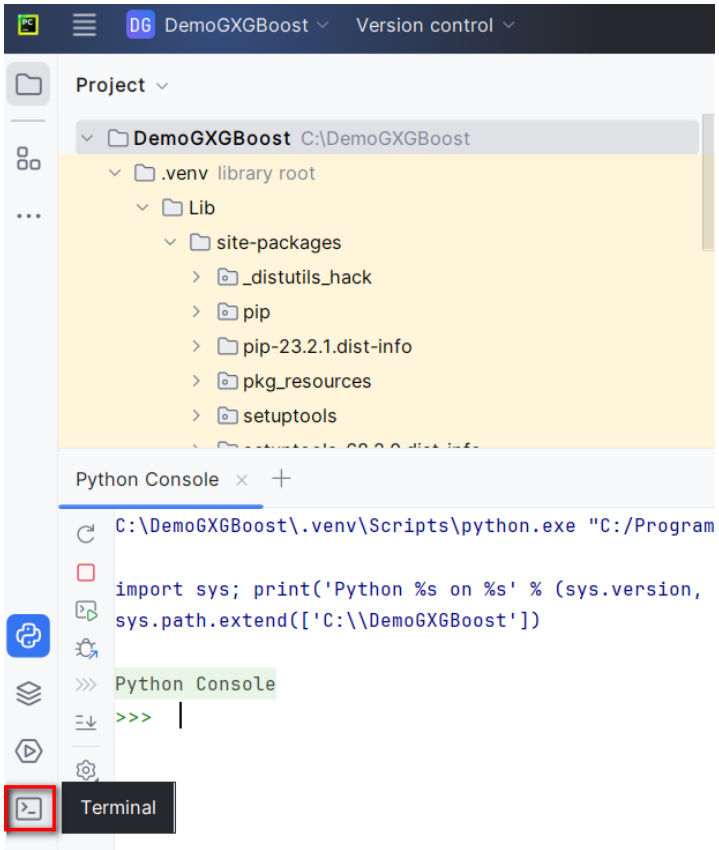


Currently, the `.venv` has only the basic libraries installed (within Lib/site-packages).

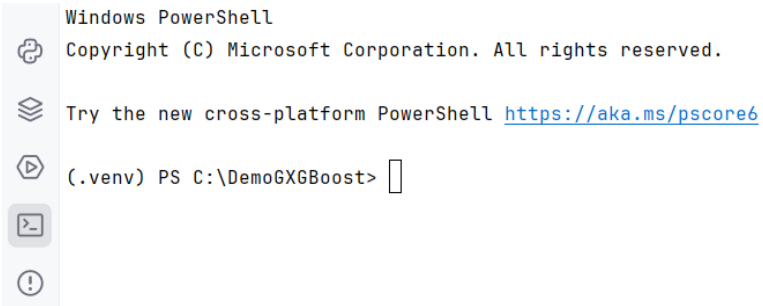


Install geoxgboost library and dependencies

- Click the button to switch from Console to Terminal



- Terminal opens



It shows the current `.venv` and the location `C:\DemoGXGBoost` where `geoxgboost` and its dependencies (additional libraries needed for `geoxgboost` to operate) will be installed.

- Install the library by typing:

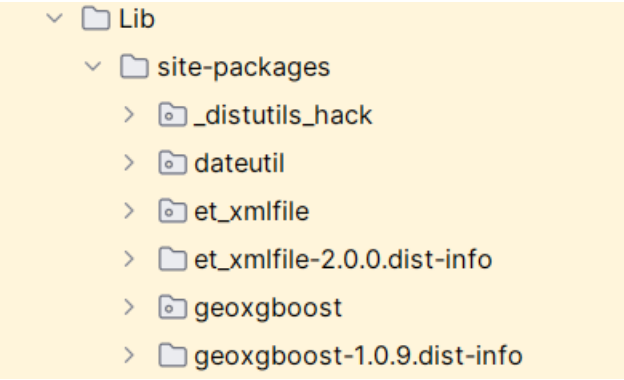
```
(.venv) PS C:\DemoGXGBoost> pip install geoxgboost
```

The dependencies are automatically downloaded and installed. The following dependencies are installed: `numpy, pandas, scikit-learn, scipy, xgboost, openpyxl.`

These libraries can be also installed manually. For example, to install openpyxl:

```
(.venv) PS C:\DemoGXGBoost> pip install openpyxl
```

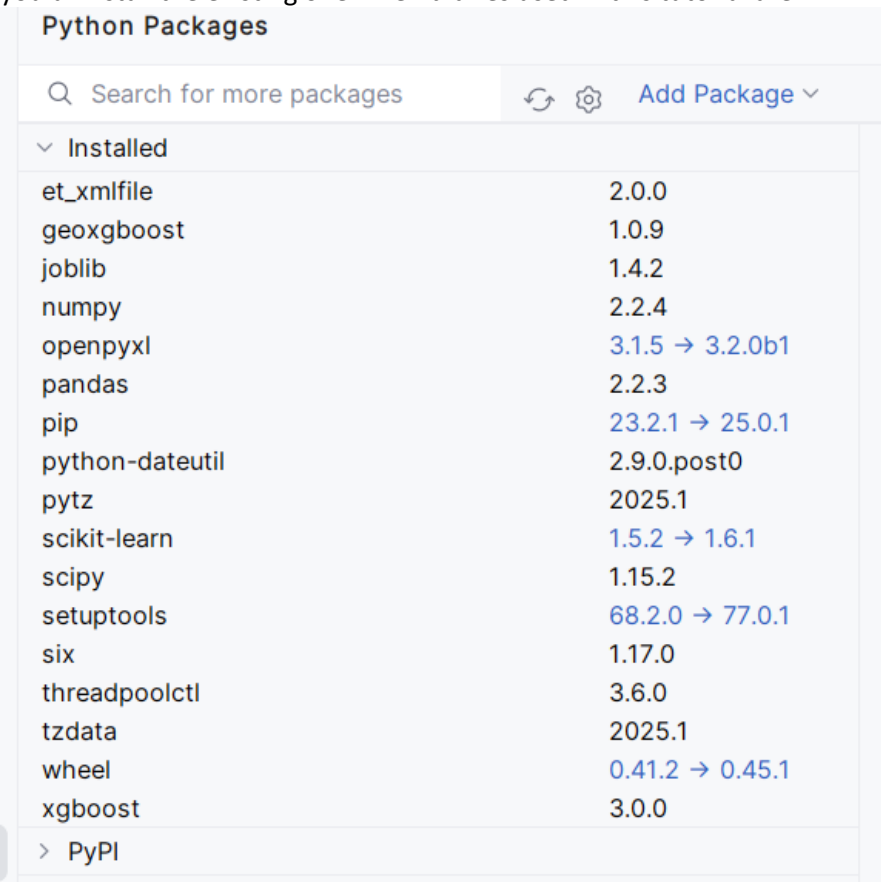
`geoxgboost` is installed within the site-packages folder (here the version installed is 1.0.9)



IMPORTANT: Uninstall `scikit-learn` and install version 1.5.2
`Scikit-learn` version 1.6 modified the API around its "tags", and this causes an error with the fit method on RandomizedSearchCV object.

```
(.venv) PS C:\DemoGXGBoost> pip uninstall -y scikit-learn
(.venv) PS C:\DemoGXGBoost> pip install scikit-learn==1.5.2
```

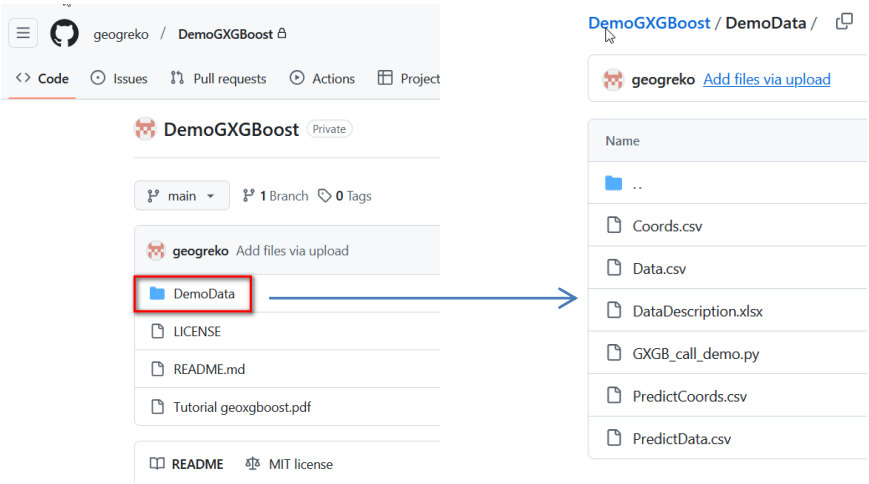
Note: This tutorial utilizes Python 3.10 and the packages listed below. Be aware that updates to these packages may cause compatibility issues with XGBoost, and consequently, geoxgboost. In case of errors, carefully examine the error message. Downgrading the conflicting package (e.g., scikit-learn) to a known working version may resolve the issue. Prior to installing an older version, ensure you uninstall the existing one. The libraries used in this tutorial are:



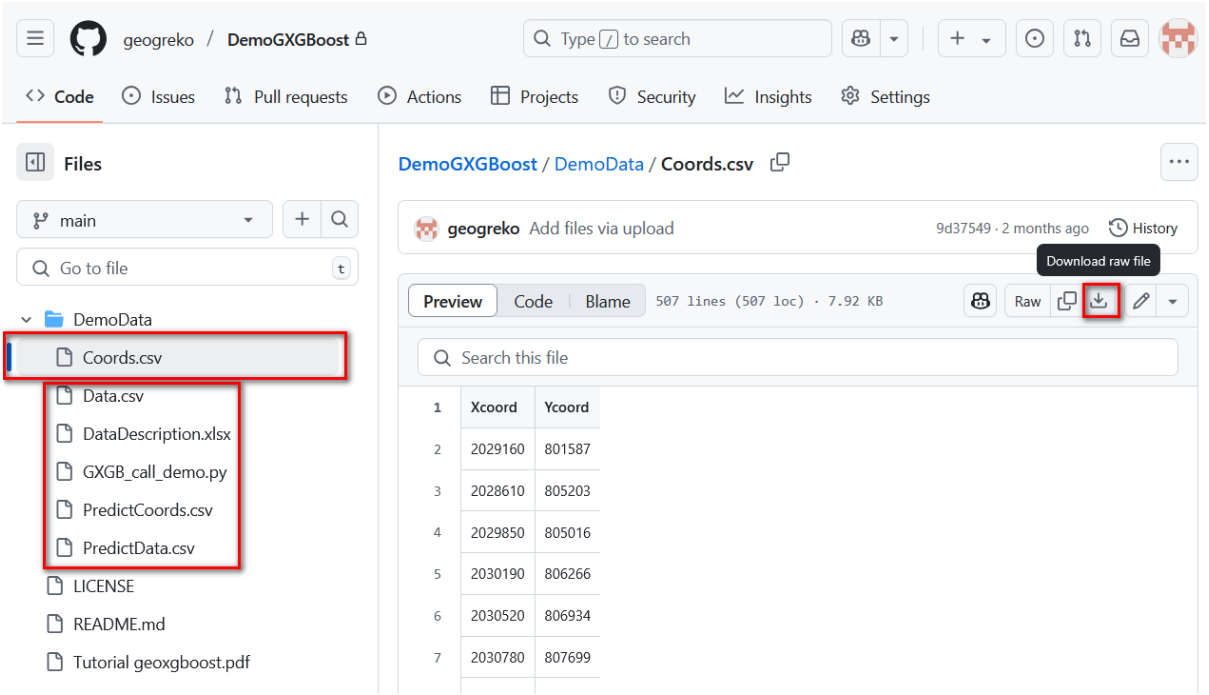
Download demo data

Download the files in the DemoData folder from https://github.com/geogreko/DemoGXGBoost

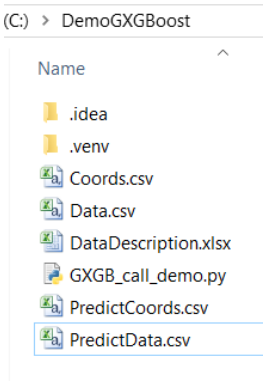
DemoGXGBoost includes the following files:



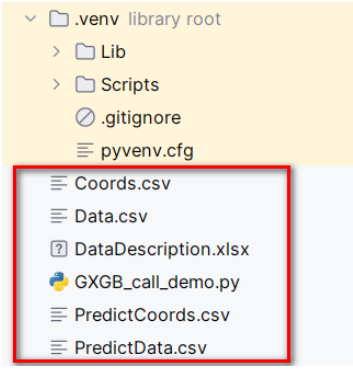
Download the six files one by one to `C:\DemoGXGBoost`
For example, click `Coords.csv` and then `Download raw file`



The folder should look like this:



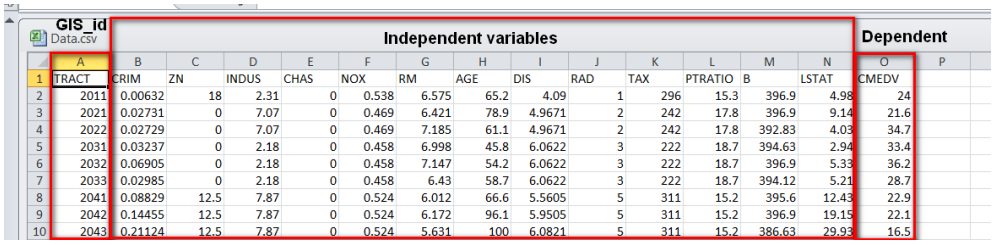
The PyCharm project folder should look like this:



DataDescription.xlsx: Contains the data dictionary, the source, and the **Boston housing dataset.**

Coords.csv: Includes the coordinates of the spatial units.

Data.csv: Includes the dependent and independent variables.



GIS_id Data.csv	Independent variables													Dependent
A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
TRACT	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO	B	LSTAT	CMEDV
2011	0.00632	18	2.31	0	0.538	6.575	65.2	4.09	1	296	15.3	396.9	4.98	24
2021	0.02731	0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8	396.9	9.14	21.6
2022	0.02729	0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8	392.83	4.03	34.7
2031	0.03237	0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7	394.63	2.94	33.4
2032	0.06905	0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7	396.9	5.33	36.2
2033	0.02985	0	2.18	0	0.458	6.43	58.7	6.0622	3	222	18.7	394.12	5.21	28.7
2041	0.08829	12.5	7.87	0	0.524	6.012	66.6	5.5605	5	311	15.2	395.6	12.43	22.9
2042	0.14455	12.5	7.87	0	0.524	6.172	96.1	5.9505	5	311	15.2	396.9	19.15	22.1
2043	0.21124	12.5	7.87	0	0.524	5.631	100	6.0821	5	311	15.2	386.63	29.93	16.5

PredictCoords.csv: Contains the coordinates of the spatial units where the prediction will occur.

PredictData.csv: Contains the values of the independent variables referring to the spatial units in which the prediction will take place.

GXGB_call_demo.py: Demo Python file for analysing the Boston housing dataset.

Run the code on the demo data

Note: Given the stochastic nature of the algorithm and/or evaluation procedure, or differences in numerical precision, results may vary.

- On PyCharm double-click `GXGB_call_demo.py`

Step 1 Import libraries and data

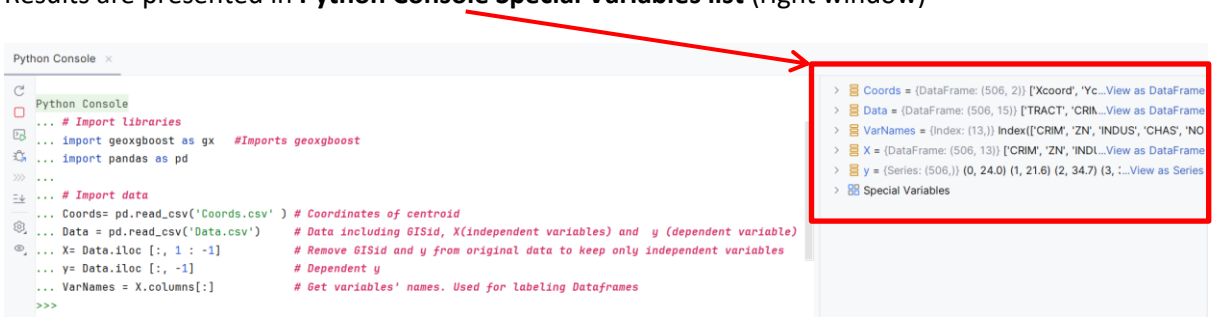
Data.csv contains a `GIS_id` field, independent variables `X`, and the dependent `y`. For this reason, it has to be cleaned to create the `DataFrame X`, containing only the independent variables, and the `DataFrame y`, containing only the dependent variable.

- Select lines 1-10 and click `Alt+Shift+E` to run the selection.

GXGB_call_demo.py

```
1   ## Step 1 Import libraries and data
2   # Import libraries
3   import geoxgboost as gx    #Imports geoxgboost
4   import pandas as pd
5   # Import data
6   Coords= pd.read_csv('Coords.csv' ) # Coordinates of centroid
7   Data = pd.read_csv('Data.csv')     # Data including GISid, X(independent var
8   X= Data.iloc [:, 1 : -1]           # Remove GISid and y from original data t
9   y= Data.iloc [:, -1]               # Dependent y
10  VarNames = X.columns[:]            # Get variables' names. Used for labeling
11
```

Results are presented in **Python Console Special Variables list** (right window)



Step 2 Hyperparameter tuning.

- Select lines in Step 2 and run the code by `Alt+Shift+E`.

```
## Step 2 Hyper parameter tuning. Define initial hyperparameters for inner loop
params= {
    'n_estimators':100,      #default is 100
    'learning_rate':0.1,     #default is 0.3
    'max_depth':6,           #default is 6
    'min_child_weight':1,    #default is 1
    'gamma':0,               #default is 0
    'subsample':0.8,         #default is 1
    'colsample_bytree':0.8, #default is 1
    'reg_alpha':0,           #default is 0
     'reg_lambda':1,         #default is 1
    }
# Define search space for hyperparameters of inner loop. A maximum of 3 hyperparameters can be tunec
Param1=None; Param2=None; Param3=None  # Set hyperparameters to None to avoid overlapping if the fur
Param1_Values = []; Param2_Values = []; Param3_Values = []
# Set hyperparameters and values according to the problem. Select and deselect for one or more hyper
Param1='n_estimators'
Param1_Values = [100, 200, 300, 500]
Param2='learning_rate'
Param2_Values = [0.1, 0.05,0.01]
Param3='max_depth'
Param3_Values = [2,3,5,6]
#Create grid
param_grid= gx.create_param_grid(Param1,Param1_Values,Param2,Param2_Values,Param3,Param3_Values)
```

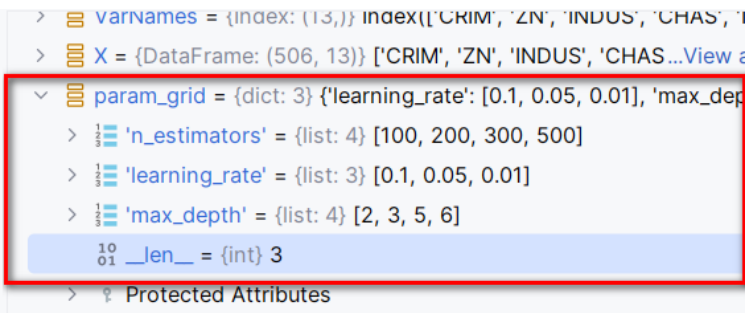
`Params` are the initial hyperparameter values (either user-defined or by default values).

Up to three hyperparameters can be tuned concurrently. To tune only two hyperparameters, comment on `Param3` and `Param3_values` (add the pound sign `#` in front of lines 33 and 34).

`Param1`: first hyperparameter name e.g., `'n_estimators'`
`Param1_Values`: values for search e.g., `[100, 200, 300, 500]`
`Param2`: second hyperparameter name e.g., `'learning_rate'`. Default=None.
`Param2_Values`: values for search e.g., `[0.1, 0.05,0.01]`. Default=None.
`Param3`: third hyperparameter name e.g., `'max_depth'`. Default=None.
`Param3_Values`: values for search e.g., `[2,3,5,6]`. Default=None.

Any tree booster hyperparameter available in XGBoost can be applied here. A complete list of available tree booster hyperparameters can be found at https://xgboost.readthedocs.io/en/stable/parameter.html#parameters-for-tree-booster

The output `param_grid` is presented in the Console Special Variables list.



Step 3 Nested CV to tune hyperparameters

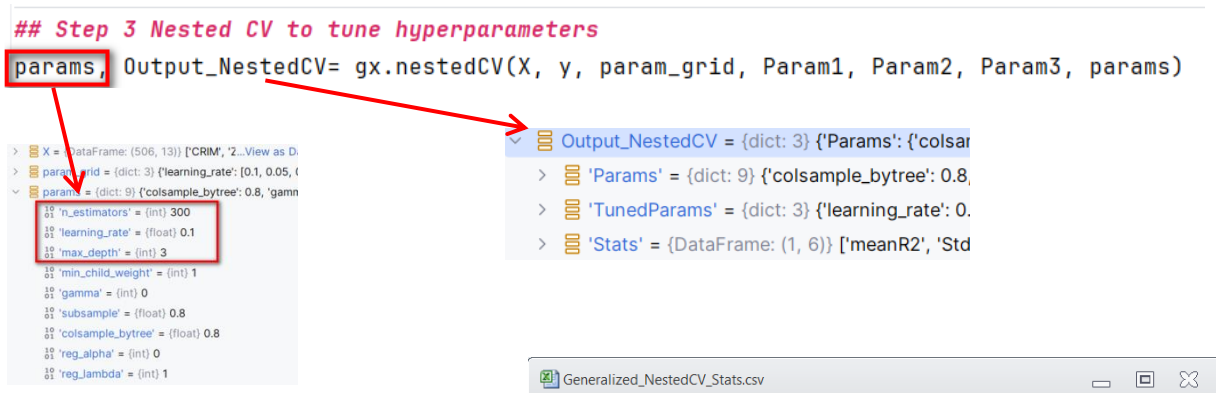
```
## Step 3 Nested CV to tune hyperparameters
params, Output_NestedCV= gx.nestedCV(X, y, param_grid, Param1, Param2, Param3, params)
```

- Run Step 3 to get the optimized hyperparameters and the model's generalization error. Run code by `Alt+Shift+E`

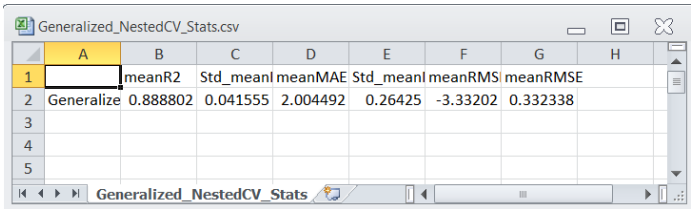
Console output:

```
>>> params, Output_NestedCV= gx.nestedCV(X, y, param_grid, Param1, Param2, Param3, params)
=================Nested CV process=============================================
Tuning with 3 hyperparameters
>Count=1,  R2=0.919, RMSE=-3.774, MAE=2.220,cfg={'learning_rate': 0.05, 'max_depth': 3, 'n_estimators':
Tuning with 3 hyperparameters
>Count=2,  R2=0.835, RMSE=-2.953, MAE=2.241,cfg={'learning_rate': 0.1, 'max_depth': 3, 'n_estimators':
Tuning with 3 hyperparameters
>Count=3,  R2=0.920, RMSE=-3.681, MAE=1.681,cfg={'learning_rate': 0.05, 'max_depth': 6, 'n_estimators':
Tuning with 3 hyperparameters
>Count=4,  R2=0.841, RMSE=-3.183, MAE=2.199,cfg={'learning_rate': 0.1, 'max_depth': 3, 'n_estimators':
Tuning with 3 hyperparameters
>Count=5,  R2=0.929, RMSE=-3.069, MAE=1.682,cfg={'learning_rate': 0.1, 'max_depth': 3, 'n_estimators':
=================Nested CV results for hyperparamtre tuning=================
Generalization error: mean-R2 (stdev): 0.889 (0.042)
Mean MAE: 2.004 (0.264)
Mean RMSE: -3.332 (0.332)
Best params taken at model with minimum RMSE at count: 2
                        meanR2 Std_meanR2  ... meanRMSE  meanRMSE
Generalized Nested CV  0.888802   0.041555  ... -3.33202  0.332338
```

Metrics for every inner loop and the generalization error of the output loop are presented. The output `params` and `Output_NestedCV` are presented in the Console Special Variables list. `params` include the initial hypermeter values updated with the optimized hyperparameter values. `Output_NestedCV` contains: `Prams values`, `TunedParams` (only the fine-tuned hyperparameters), and `Stats`. `Stats` stores the generalization statistics of the nested cv process: `mean R2`, `mean MAE`, `mean RMSE`, and their standard deviations (`std`). `Stats` is also saved as `csv` file.



The `csv` file containing `Stats` of the nested cv:



File is exported in `csv` format as `Generalized_NestedCV.csv` and is saved in `C:\DemoGXGBoost` (unless a different path is specified in the `path_save` property).

Step 4 GlobalXGBoost model

```
## Step 4 GlobalXGBoost model
Output_GlobalXGBoost=gx.global_xgb(X,y,params)
```

- Run Step 4 for the Global XGBoost model by `Alt+Shift+E`

Console output:

```
=================XGBoost (global) evaluation results ========================
Global feature importance:
   Imp_CRIM    Imp_ZN  Imp_INDUS  ...  Imp_PTRATIO     Imp_B  Imp_LSTAT
0  0.041723  0.003115   0.024551  ...     0.055827  0.011992   0.394174

[1 rows x 13 columns]
Test R2: 86.75%
==================Stats and importance=========================================
            R2Pred   MAEPred   RMSPred  ... Imp_PTRATIO     Imp_B Imp_LSTAT
GlobalXGB  0.95838  0.966699  1.871395  ...    0.055827  0.011992  0.394174

[1 rows x 19 columns]
Results have been saved in xlsx format at the specified path
==============================================================================
```

```
Output_GlobalXGBoost = {dict: 3} {'Importance': ['Imp_CRIM ... View
  'Predictions' = {DataFrame: (506, 3)} ['y', 'I...View as DataFrame
  'Stats' = {DataFrame: (1, 19)} ['R2Pred', 'M...View as DataFrame
  'Importance' = {DataFrame: (1, 13)} ['Imp_C...View as DataFrame
  __len__ = {int} 3
  Protected Attributes
Output_NestedCV = {dict: 3} {'Params': {'colsample_bytree': ... View
Param1 = {str} 'n_estimators'
Param1_Values = {list: 4} [100, 200, 300, 500]
Param2 = {str} 'learning_rate'
Param2_Values = {list: 3} [0.1, 0.05, 0.01]
Param3 = {str} 'max_depth'
```

Output: Global feature importance and evaluation metrics are presented in the console, along with the xlsx file.

GlobalXGB.xlsx — Statistics / Feature Importance

	R2Pred	MAEPred	RMSPred	R2test	MAEtest	RMSETest	Imp_CRIM	Imp_ZN	mp_INDUS	Imp_CHAS	Imp_NOX	Imp_RM	Imp_AGE	Imp_DIS	Imp_RAD	Im
GlobalXGB	0.95838	0.966699	1.871395	0.867454	2.135297	3.180311	0.041723	0.003115	0.024551	0.036681	0.055955	0.258167	0.014539	0.051118	0.013169	0

Note:
Pred metrics refer to the full set (after training, prediction using all data). Used only for reference. | Test metrics refer to the test set. | Imp_ refers to feature importance

Citation:
Grekousis G. (2025). Geographical-XGBoost: A new ensemble model for spatially local regression based on gradient-boosted trees. Journal of Geographical Systems, https://doi.org

Stats | Predict

`Predict` is in a separate worksheet having: `y` (true value of `y`), `GM_yPred` (global model `y` prediction), and `GM_Res` (global model residuals).

GlobalXGB.xlsx — Predict

	y	GM_yPred	GM_Res
2	24	24.19805	-0.19805
3	21.6	21.03201	0.567991
4	34.7	36.2447	-1.5447
5	33.4	34.14556	-0.74556
6	36.2	36.08273	0.117271
7	28.7	28.40969	0.290313
8	22.9	22.38103	0.518967
9	22.1	20.75484	1.345163
10	16.5	16.32705	0.172953
11	18.9	18.40004	0.49996
12	15	16.04512	-1.04512
13	18.9	19.84567	-0.94567

Stats | Predict

File is exported in `xlsx` format as `GlobalXGB.xlsx` and is saved in `C:\DemoGXGBoost` (unless a different path is specified in the `path_save` property).

Step 5 Optimize Bandwidth

```
## Step 5 Optimize Bandwidth
bw= gx.optimize_bw(X,y, Coords, params, bw_min=77, bw_max=83,step=1, Kernel='Adaptive', spatial_weights=True)
```

- Run Step 5 to find the optimal bandwidth value by `Alt+Shift+E`

Optimal bandwidth can be computationally expensive. Instead of an exhaustive search for an extensive range of values (e.g., 30 to 100 number of nearest neighbors), use a large incremental step (e.g., 5 or 10) and then search around the bandwidth value that minimizes the cross-validation criterion by reducing the step size.

Wait… this could take several minutes…

Console output:

```
=================Calculating optimal bandwidth==============================
Calculation with spatial weights
Adaptive Kernel used
Calculating bw= 77, with bw_max=83 and step of 1
bw= 77, with CV= 4021.529
Calculating bw= 78, with bw_max=83 and step of 1
bw= 78, with CV= 4079.879
Calculating bw= 79, with bw_max=83 and step of 1
bw= 79, with CV= 3947.695
Calculating bw= 80, with bw_max=83 and step of 1
bw= 80, with CV= 3922.370
Calculating bw= 81, with bw_max=83 and step of 1
bw= 81, with CV= 3960.386
Calculating bw= 82, with bw_max=83 and step of 1
bw= 82, with CV= 3969.066
Calculating bw= 83, with bw_max=83 and step of 1
bw= 83, with CV= 4046.078
Best bandwidth value: 80 at min CV= 3922.370.
Results have been saved in csv format at the specified path
==============================================================================
```

The function calculates `R2`, `MAE`, `RMSE`, and `CV` (cross-validation criterion). Optimal bandwidth (`BW`) is the value that minimizes `CV`.

BW_results.csv

	A	B	C	D	E	F
1		BW	R2	MAE	RMSE	CV
2	0	77	0.905549	2.004796	2.819164	4021.529
3	1	78	0.904178	2.001908	2.839543	4079.879
4	2	79	0.907283	1.986634	2.793165	3947.695
5	3	80	0.907877	1.966615	2.784191	3922.37
6	4	81	0.906985	1.972536	2.797651	3960.386
7	5	82	0.906781	1.973813	2.800715	3969.066
8	6	83	0.904972	1.981142	2.827756	4046.078
9						

File is exported in `csv` format as `BW_results.csv` and is saved in `C:\DemoGXGBoost` (unless a different path is specified in the `path_save` property).

Step 6 GXGB (Geographical-XGBoost)

```
## Step 6 GXGB (Geographical-XGBoost)
Output_GXGB_LocalModel= gx.gxgb(X,y,Coords, params,bw=bw, Kernel='Adaptive', spatial_weights=True, alpha_wt_type='varying', alpha_wt=0.5)
```

The above example uses spatial weights (`spatial_weights=True`) and a varying alpha weight (`alpha_wt=0.5`), leading to an ensemble solution. The console should look like this.

```
Python Console
>>> Output_GXGB_LocalModel= gx.gxgb(X,y,Coords, params,bw=bw, Kernel='Adaptive', spatial_weights=True, alpha_wt_type='varying', alpha_wt=0.5)
================Calculating Geographical XGBOOST===========================
Calculating i= 1 from a total of 506
Adaptive Kernel used with 80 neighbours
Calculation with spatial weights
Calculation with ensemble and varying alpha_wt
a= 1.00, b=0.00
Calculating i= 2 from a total of 506
Adaptive Kernel used with 80 neighbours
Calculation with spatial weights
Calculation with ensemble and varying alpha_wt
a= 1.00, b=0.00
Calculating i= 3 from a total of 506
Adaptive Kernel used with 80 neighbours
```

The output is saved to various tables and is exported in xlsx format

```
Output_GXGB_LocalModel = {dict: 6} {'Params': ['Value'] [n_estimators        ... View
  'Params' = {DataFrame: (19, 1)} ['Value'] [n_estimators      ...View as DataFrame
  'Stats' = {DataFrame: (1, 12)} ['R2_Pred', 'MAE_Pred', 'RMS...View as DataFrame
  'Prediction' = {DataFrame: (506, 25)} ['IDS', 'y', 'LM_yPred...View as DataFrame
  'alpha_wt' = {list: 506} [1, 1, 0.5, 1, 0.5, 0.5, 0.5, 1, 1, 0.5, 1, 0.5, 1, 0.5, ... View
  'y_G_hat' = {list: 506} [np.float32(25.901796), np.float32(20.969429), n... View
  'bestLocalModel' = {list: 506} [XGBRegressor(base_score=None, boost... View
  __len__ = {int} 6
```

GXGB.xlsx

	A	B	C	D	E	F	G	H	I	J	K	L	M
											Ensemble Modiel		
1		R2_Pred	MAE_Pred	RMS_Pred	R2_oob	MAE_oob	RMS_oob	R2oobGl	MAEoobG	RMSEoobG	R2ens	MAEens	RMSEens
2	GXGB	0.999993	0.017056	0.024637	0.907877	1.966615	2.784191	0.8826	2.106304	3.143037	0.9351	1.596466	2.336886
3													
4		Value											
5	n_estimators	300											
6	learning_rate	0.1											
7	max_depth	3											
8	min_child_weight	1											
9	gamma	0											
10	subsample	0.8											
11	colsample_bytree	0.8											
12	reg_alpha	0											
13	reg_lambda	1											
14	Spatial Units	506											
15	Features	13											
16	Kernel	Adaptive											
17	Bandwidth	80											
18	Spatial Weights	TRUE											
19	Alpha Weight type	varying											
20	Alpha Weight value	0.5											
21	Feature Importance	gain											
22	Test Size	0.3											
23	Seed	7											

Hyperparameters and settings

Stats | LocalModels

A short description for every variable is also provided:

	Description
IDS	Spatial unit id
y	y_true
LM_yPred	Local model prediction including central point
LM_yOOB	Local model prediction excluding central point
LM_ResOOB	Local model OOB residuals
LMRsqr	Local model Rsqr (oob)
LM_Best_score(RMSE)	Local model best score (oob)
alpha_wt	alpha weight value
yGhat	Prediction of the global model excluding central point
y_ensemble	prediction of y through ensemble
Imp_	Feature local importance
MaxImportance	Maximum feature importance
MaxFeatureID	Feature with max importance

Click the Local Models worksheet to get the detailed results per local model

GXGB.xlsx

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y
1	IDS	y	LM_yPred	LM_yOOB	M_ResOO	LMRsqr	est_score(	alpha_wt	yGhat	_ensembl	Imp_CRIM	Imp_ZN	Imp_INDU	Imp_CHAS	Imp_NOX	Imp_RM	Imp_AGE	Imp_DIS	Imp_RAD	Imp_TAX	p_PTRAT	Imp_B	Imp_LSTAT	MaxImportance	MaxFeatureID
2	0	24	23.9989	25.39	-1.38996	0.67397	-3.01907	1	26.7528	25.39	0.00937	0	0.00731	0	0.01935	0.12762	0.157	0.05724	0.03935	0.1295	0.09191	0.09455	0.26681876	0.266818762	13
3	1	21.6	21.5701	20.5829	1.01714	0.74426	-2.34065	1	20.1928	20.5829	0.05783	0.01236	0.10917	0	0.02554	0.10034	0.06237	0.00649	0.00353	0.19262	0.01455	0.02905	0.38613787	0.386137873	13
4	2	34.7	34.7046	28.2259	6.47412	0.71006	-2.27739	0.5	35.4689	31.8474	0.06686	0.0039	0.29092	0	0.00651	0.10099	0.04219	0.00413	0.01432	0.14529	0.00391	0.03211	0.28887665	0.290922493	3
5	3	33.4	33.4287	33.6966	-0.29665	0.71682	-2.41867	1	36.9831	33.6966	0.06789	0.00669	0.10964	0	0.01122	0.20107	0.09348	0.00809	0.00265	0.13999	0.01764	0.03476	0.30688199	0.306881994	13
6	4	36.2	36.1862	30.1202	6.07984	0.84173	-1.96917	0.5	33.5502	31.8352	0.18637	0.00133	0.06087	0	0.014	0.12891	0.0708	0.00704	0.01043	0.05624	0.03	0.03778	0.39623669	0.396236688	13

File is exported in `xlsx` format as:

```
GXGB.xlsx      (if spatial_weights=True,  alpha_wt_type= 'varying')
LW_GXGB.xlsx   (if spatial_weights=True,  alpha_wt_type= 'fixed', alpha_wt=1)
L_GXGB.xlsx    (if spatial_weights=False, alpha_wt_type= 'fixed', alpha_wt=1)
```

The file is saved in `C:\DemoGXGBoost` (unless a different path is specified in the `path_save` property).

Step 7 Predict (unseen data)

```
## Step 7 Predict (unseen data)
# Input data to predict
DataPredict = pd.read_csv('PredictData.csv')
CoordsPredict= pd.read_csv('PredictCoords.csv')
# predict
Output_PredictGXGBoost= gx.predict_gxgb(DataPredict, CoordsPredict, Coords, Output_GXGB_LocalModel, alpha_wt = 0.5, alpha_wt_type = 'varying')
```

Run step 7 to use `geoxgboost` for prediction in unseen data. It needs a trained model that already exists. Have in mind to use local trained models with the same parameters as those specified in the function. For example, if `Output_GXGB_LocalModel` is trained with `alpha_wt=0.5` and `alpha_wt_type='varying'`, the same parameters should be used here.

Console output:

```
=============Predict Geographical-XGBoost ==========================
      Y_PRED
0  28.433449
1  24.895836
2  24.576115
3  22.337307
4  20.590096
5  19.849297
6  17.208260
7  15.864753
8  18.120311
Results have been saved in xlsx format at the specified path
=====================================================================
```

```
Output_PredictGXGBoost = {DataFrame: (9, 1)} ['Y_PRED'] [...View as DataFrame
  T = {DataFrame: (1, 9)} [0, 1, 2, 3, 4, 5, 6, 7, 8] [Y_PRED ...View as DataFrame
  Y_PRED = {Series: (9,)} (0, 28.433448791503906) (1, 24.895...View as Series
```

File is exported in `csv` format as `Predict_results.csv` and is saved in `C:\DemoGXGBoost` (unless a different path is specified in the `path_save` property).

Note This tutorial demonstrates a standard workflow for G-XGBoost, focusing on achieving near-optimal performance for the provided dataset and is not exhaustive. Readers are encouraged to experiment with various hyperparameter and function parameter settings on their own datasets.

How to cite: Grekousis G. (2025). Geographical-XGBoost: A new ensemble model for spatially local regression based on gradient-boosted trees. *Journal of Geographical Systems*, https://doi.org/10.1007/s10109-025-00465-4